

System Models

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling has generally come to mean representing the system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language
- Models are used during the requirements engineering process to help derive the requirements for a system, during the design process to describe the system to engineers implementing the system and after implementation to document the system's structure and operation.

System Models

- Developing different models to represent the system from different perspectives.
 - For example:
 1. **An external perspective**, where you model the context or environment of the system.
 2. **An interaction perspective** where you model the interactions between a system and its environment or between the components of a system.
 3. **A structural perspective**, where you model the organization of a system or the structure of the data that is processed by the system.
 4. **A behavioral perspective**, where you model the dynamic behavior of the system and how it responds to events.
-

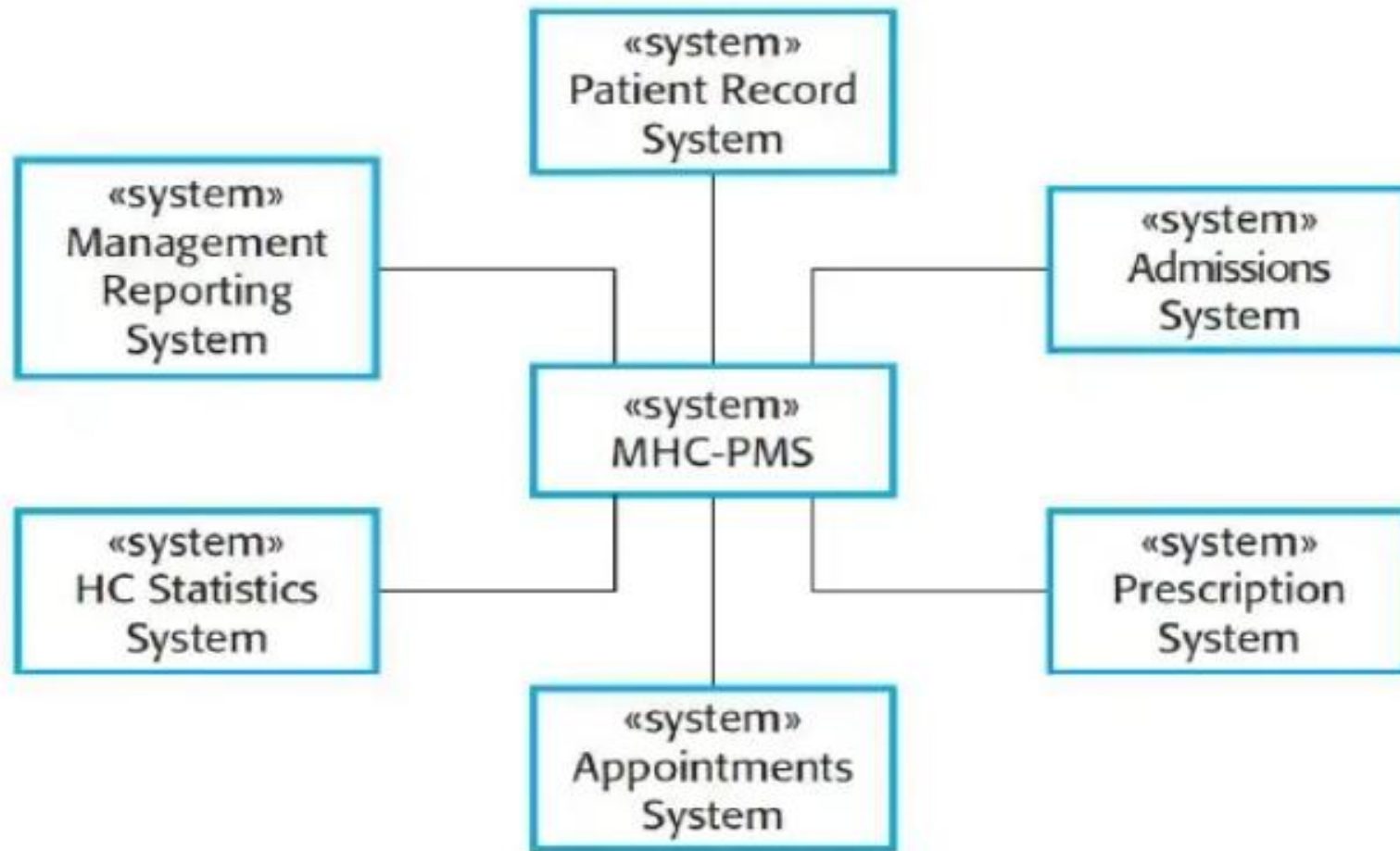
Context models

- you should decide on the system boundaries. This involves working with system stakeholders to decide what functionality should be included in the system and what is provided by the system's environment.
- You may decide that automated support for some business processes should be implemented but others should be manual processes or supported by different systems.
- You should look at possible overlaps in functionality with existing systems and decide where new functionality should be implemented.
- These decisions should be made early in the process to limit the system costs and the time needed for understanding the system requirements and design.

Context models

- For example, say you are developing the specification for the patient information system for mental healthcare.
- This system is intended to manage information about patients attending mental health clinics and the treatments that have been prescribed.
- In developing the specification for this system, you have to decide whether the system should focus exclusively on collecting information about consultations (using other systems to collect personal information about patients) or whether it should also collect personal patient information.
- The advantage of relying on other systems for patient information is that you avoid duplicating data.

Context models



Interaction models

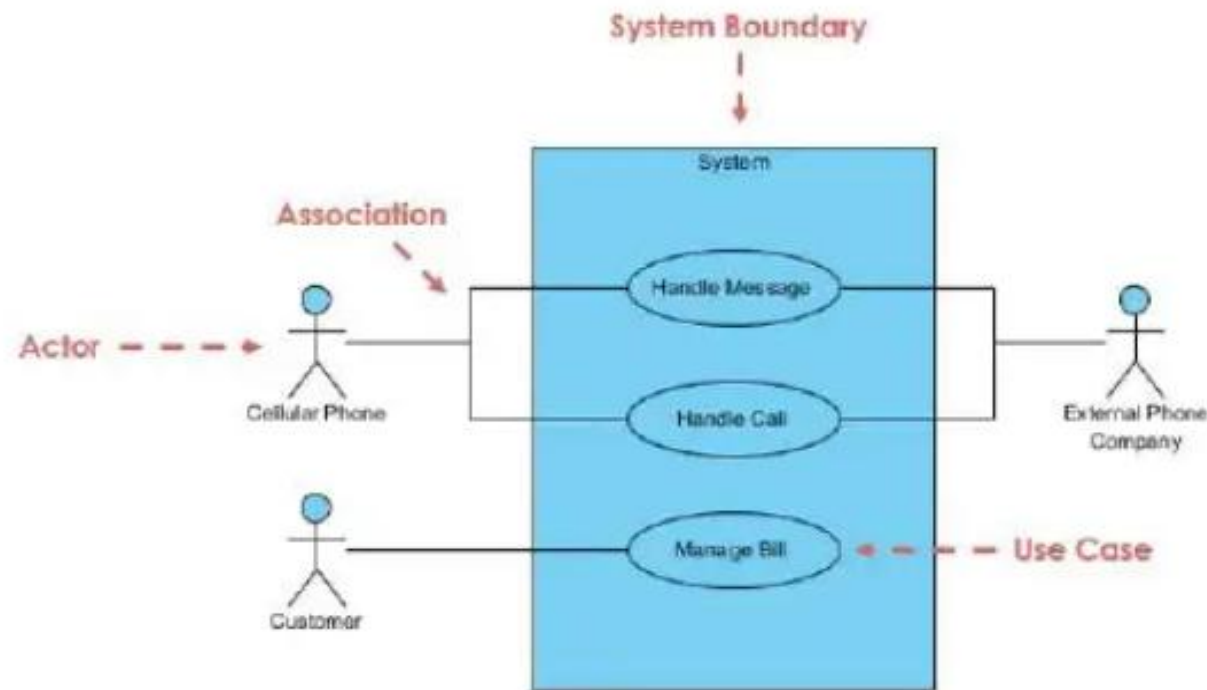
- All systems involve interaction of some kind.
- This can be user interaction, which involves user inputs and outputs, interaction between the system being developed
- other systems or interaction between the components of the system.
- Modeling user interaction is important as it helps to identify user requirements.
- Modeling system to system interaction highlights the communication problems that may arise.

Interaction models

- Two related approaches to interaction modeling:
 1. **Use case modeling**, which is mostly used to model interactions between a system and external actors (users or other systems).
 2. **Sequence diagrams**, which are used to model interactions between system components, although external agents may also be included.

Interaction models - Use case modeling

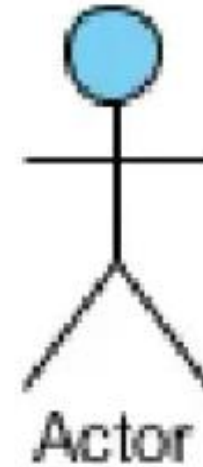
- Use case modeling was originally developed by Jacobson et al. (1993) in the 1990s



Interaction models - Use case modeling

Actor

- Someone interacts with use case (system function).
- Named by noun.
- Actor plays a role in the business
- Similar to the concept of user, but a user can play different roles
- For example:
 - A prof. can be instructor and also researcher
 - plays 2 roles with two systems
- Actor triggers use case(s).
- Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs).



Interaction models - Use case modeling

Use Case

- System function (process - automated or manual)
- Named by verb + Noun (or Noun Phrase).
- i.e. Do something
- Each Actor must be linked to a use case, while some use cases may not be linked to actors..



Interaction models - Use case modeling

Communication Link

- The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.
- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.

Interaction models - Use case modeling

Boundary of system

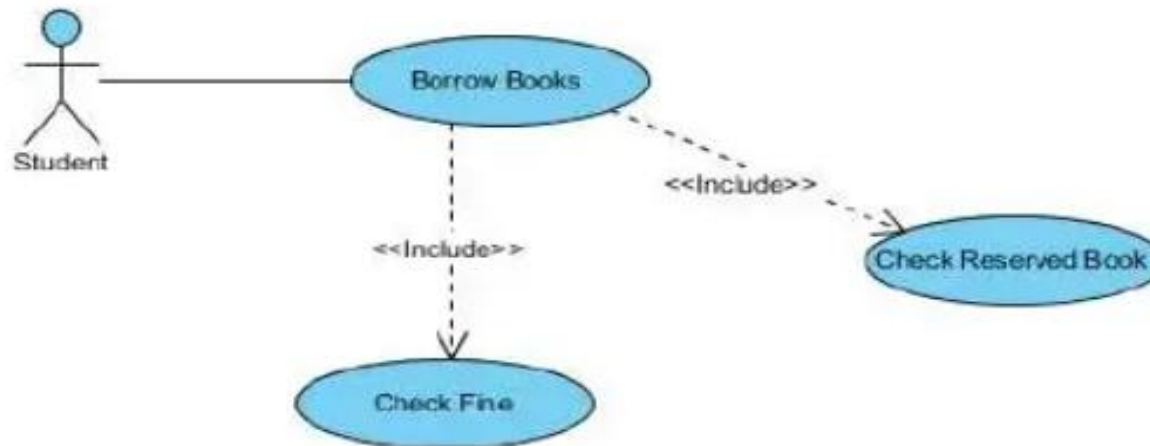
- The system boundary is potentially the entire system as defined in the requirements document.
- For large and complex systems, each module may be the system boundary.
- For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc.
- can form a system boundary for use cases specific to each of these business functions.
- The entire system can span all of these modules depicting the overall system boundary



Interaction models - Use case modeling

Include Relationship

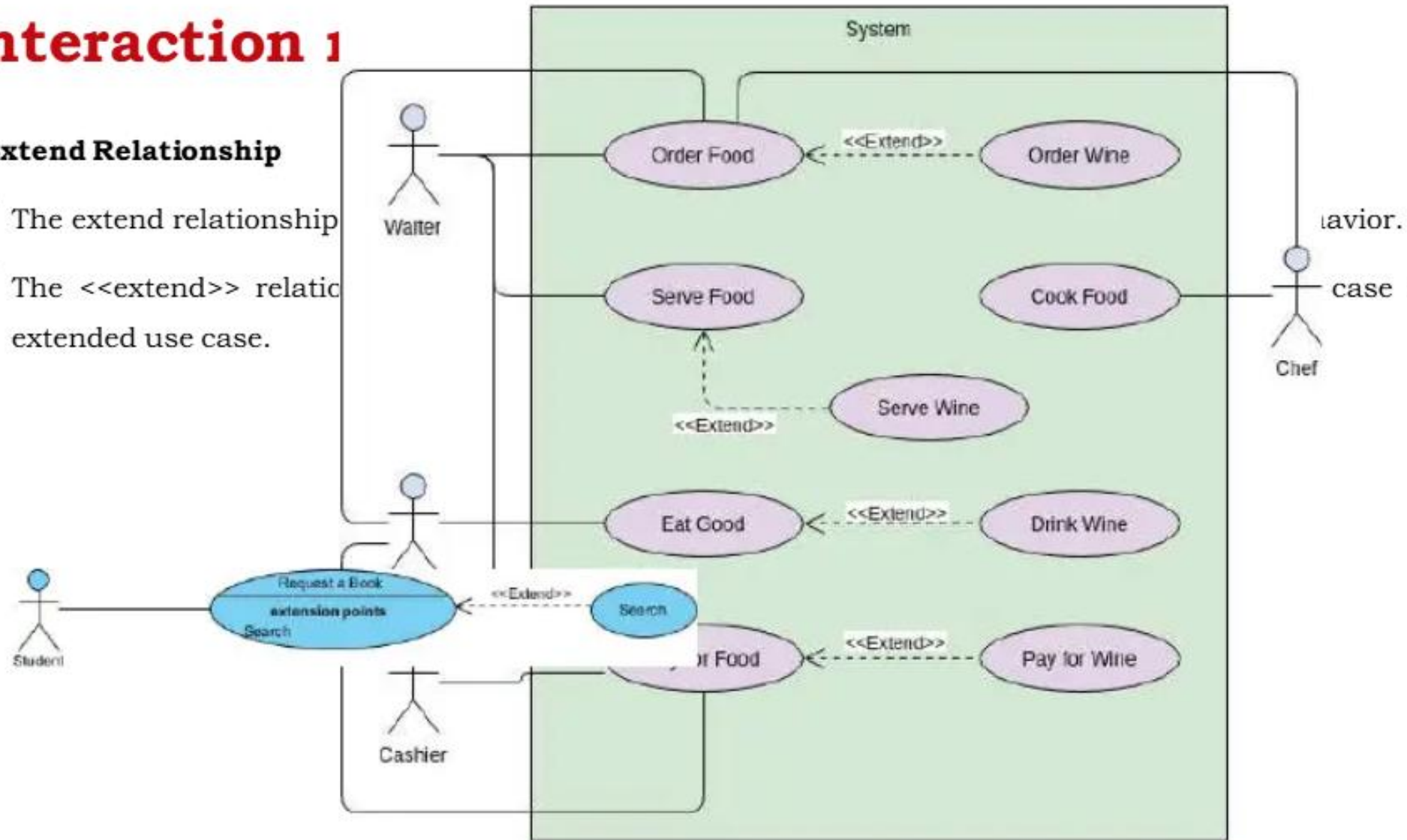
- The include relationship adds additional functionality not specified in the base use case.
- The <<Include>> relationship is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



Interaction 1

Extend Relationship

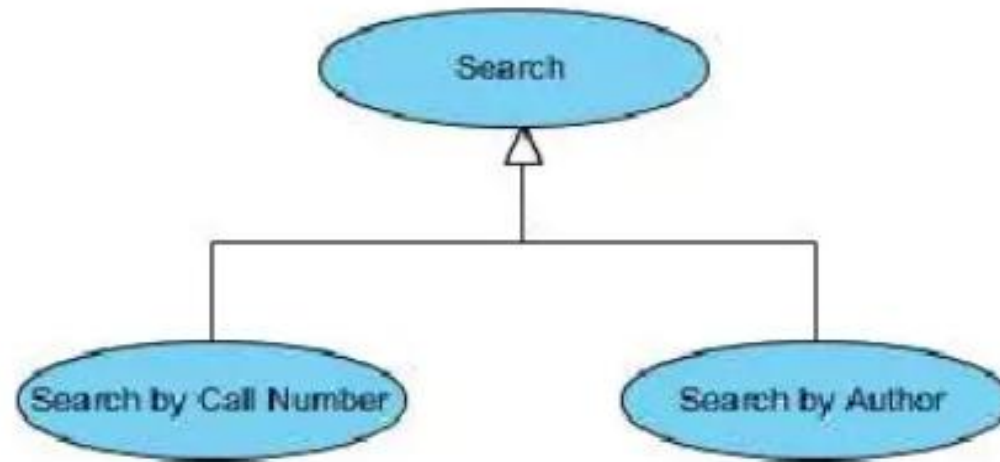
- The extend relationship
- The <<extend>> relationship is used to extend a use case in an extended use case.



Interaction models - Use case modeling

Generalization Relationship

- A generalization relationship means that a child use case inherits the behavior and meaning of the parent use case.
- The child may add or override the behavior of the parent.
- The figure below provides a use case example by showing two generalization connectors that connect between the three use cases.



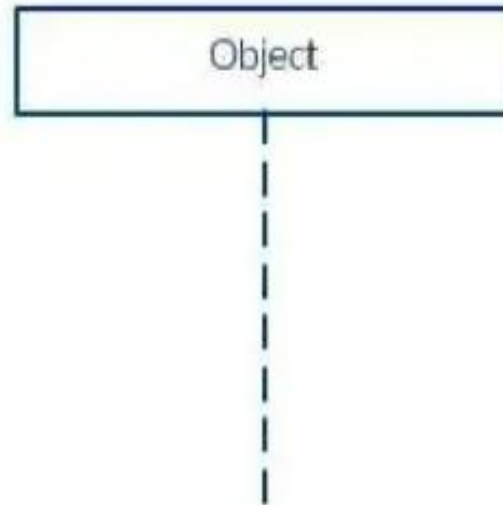
Interaction models - Sequence diagrams

- The sequence diagram represents the flow of messages in the system and is also termed as an event diagram.
- It helps in envisioning several dynamic scenarios.
- Sequence diagrams are particularly useful for depicting the dynamic behavior of a system and how different components collaborate to achieve a specific task or goal.
- They are commonly used during the design and documentation phases of software development.

Interaction models - Sequence diagrams

Key elements and concepts in a sequence diagram include:

- **Lifelines:** These represent the individual objects or components involved in the interaction. Each lifeline is typically depicted as a vertical line, and their names are often placed at the top of these lines.

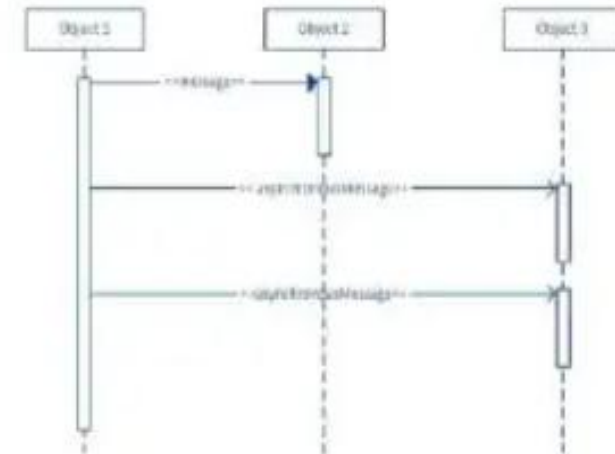


Interaction models - Sequence diagrams

Key elements and concepts in a sequence diagram include:

Messages:

- These are the horizontal arrows or lines that connect lifelines, indicating the order of messages passed between objects.
- Messages can be synchronous (blocking) or asynchronous (non-blocking) and they typically have labels that describe the action or method being invoked.
- As shown in the activation bars example, a synchronous message is used when the sender waits for the receiver to process the message and return before carrying on with another message.
- An asynchronous message is used when the message caller does not wait for the receiver to process the message and return before sending other messages to other objects within the system.

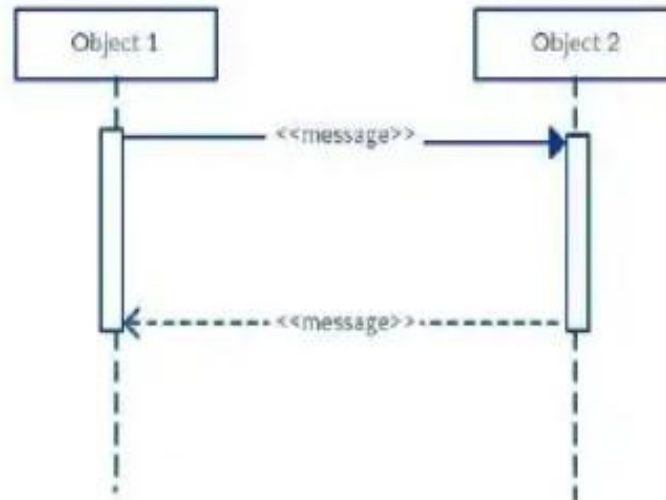


Interaction models - Sequence diagrams

Key elements and concepts in a sequence diagram include:

Return Message:

A return message is used to indicate that the message receiver is done processing the message and is returning control over to the message caller. Return messages are optional notation pieces, for an activation bar that is triggered by a synchronous message always implies a return message.

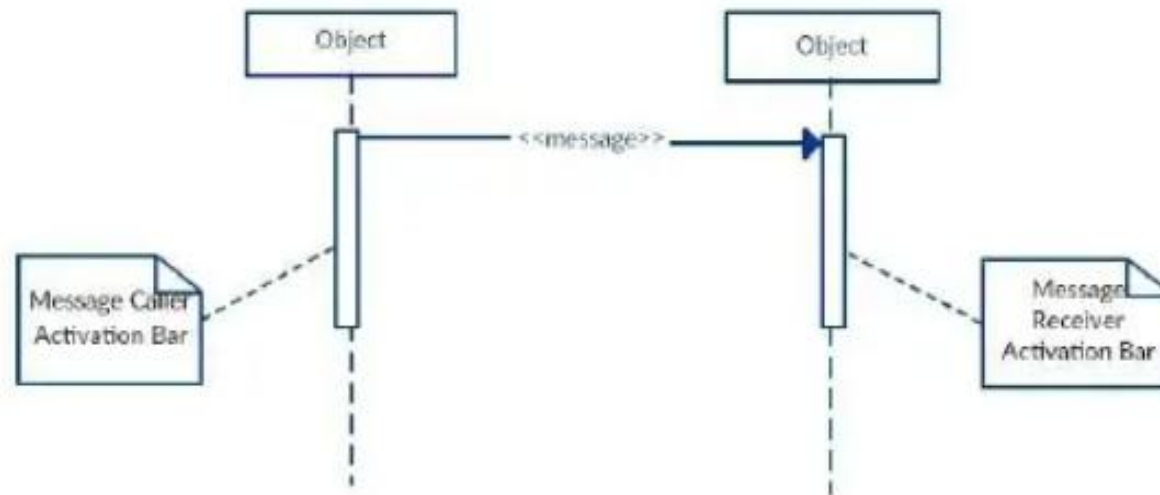


Interaction models - Sequence diagrams

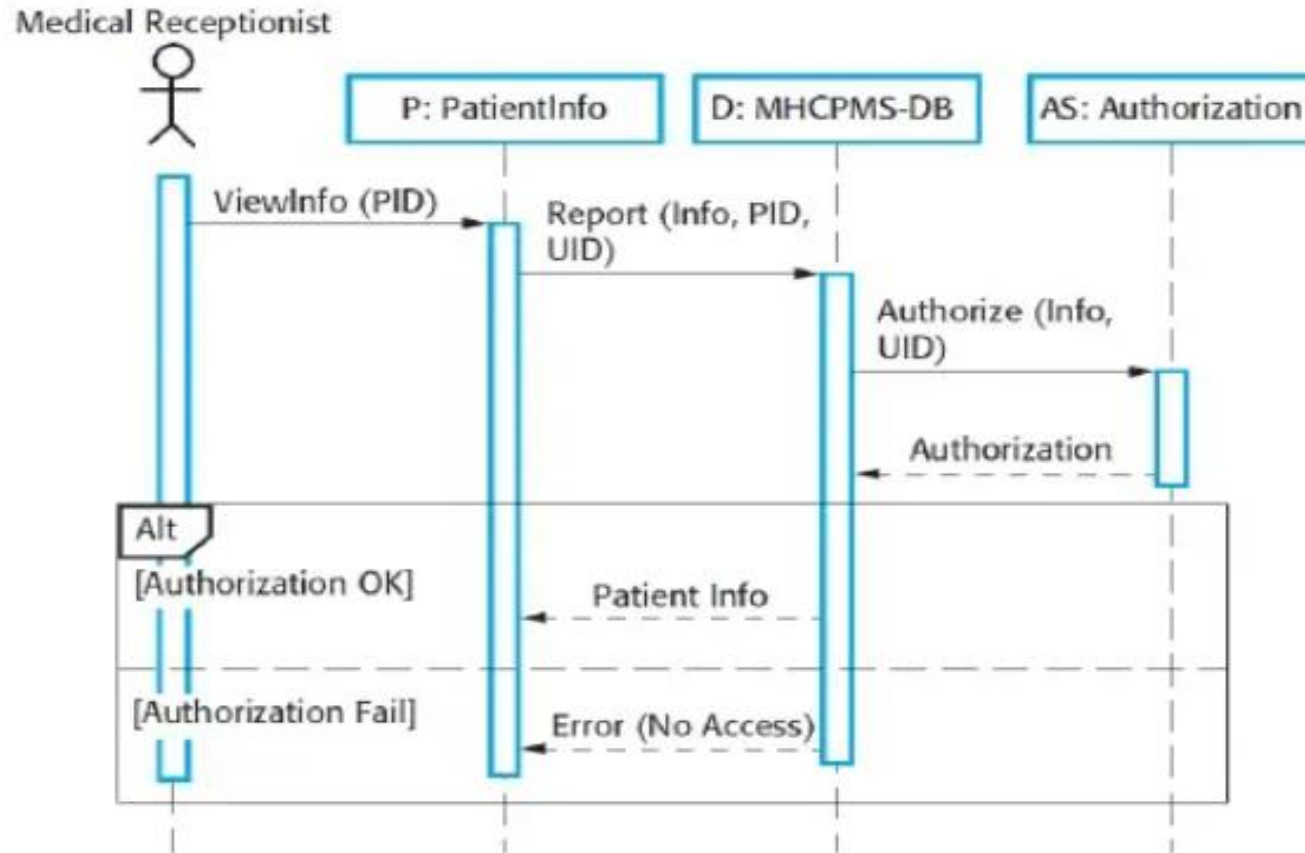
Key elements and concepts in a sequence diagram include:

Activation Bars:

- These represent the period during which an object is active and processing a message.
- They are depicted as a box or vertical rectangle on the lifeline and show the duration of the object's involvement in the interaction.



Interaction models - Sequence diagrams



Structural models

- Structural models of software **display** the organization of a system in terms of the components that make up that system and **their relationships**.
- Structural models may be static models, which show the structure of the system design or dynamic models, which show the organization of the system when it is executing.
- These are not the same things—the dynamic organization of a system as a set of interacting threads may be very different from a static model of the system components.
- **Structural models of a system is created when you are discussing and designing the system architecture.**

Structural models

- Structural UML diagrams illustrate the organization of a system by depicting its components, such as classes, objects, and packages.
- They represent the elements that make up the system and the relationships between them.

Structural models

